Student Guide: Git & GitHub for Team Projects

Table of Contents

1. Initial Setup
2. Basic Team Workflow
3. Working with Branches
4. Collaborating with Pull Requests
5. Handling Conflicts
6. Best Practices
7. Common Commands Reference

---

1. Initial Setup

First-Time Git Configuration

```bash
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Cloning Your Team's Repository

```bash
git clone https://github.com/username/repository-name.git
cd repository-name
```

---

2. Basic Team Workflow

The standard workflow follows this pattern:

1. **Pull the latest changes** (always do this before starting work)
2. **Create a new branch** for your feature/task
3. **Make your changes** and commit regularly
4. **Push your branch** to GitHub

5. **Create a pull request** for review
6. **Merge** after approval

---

3. Working with Branches

Why Branches Matter

Branches let everyone work on different features simultaneously without interfering with each other's code.

Creating and Switching Branches

```bash
# Create a new branch and switch to it
git checkout -b feature/your-feature-name

# Or using newer syntax
git switch -c feature/your-feature-name
```

Branch Naming Conventions

- `feature/login-page` - new features
- `bugfix/header-alignment` - bug fixes
- `hotfix/security-patch` - urgent fixes
- `docs/readme-update` - documentation

Checking Your Current Branch

```bash
git branch      # Shows all local branches
git branch -a   # Shows all branches including remote
```

---

4. Making Changes and Committing

The Daily Workflow

```bash
```

```
# 1. Check what branch you're on
git branch

# 2. Make sure you have latest changes
git pull origin main

# 3. Check what files you've changed
git status

# 4. Stage your changes
git add filename.txt          # Add specific file
git add .                     # Add all changed files

# 5. Commit with a descriptive message
git commit -m "Add user login validation"

# 6. Push to GitHub
git push origin feature/your-feature-name
```

Writing Good Commit Messages

Good:

- "Add email validation to signup form"
- "Fix navbar collapse on mobile devices"
- "Update README with installation steps"

Bad:

- "fixed stuff"
- "asdfasdf"
- "updates"

---

5. Collaborating with Pull Requests

Creating a Pull Request

1. Push your branch to GitHub
2. Go to your repository on GitHub
3. Click "Pull requests" → "New pull request"
4. Select your branch to merge into `main`

5. Add a clear title and description of what you changed
6. Request reviews from teammates
7. Wait for approval before merging

Reviewing Someone's Pull Request

1. Read the description to understand what changed
2. Click on "Files changed" to see the code
3. Leave comments on specific lines if you have questions
4. Approve if it looks good, or request changes if needed

---

6. Handling Merge Conflicts

Conflicts happen when two people edit the same lines of code. Don't panic!

Resolving Conflicts

```bash
# 1. Update your branch with latest main
git checkout main
git pull origin main
git checkout your-branch
git merge main

# 2. Git will tell you which files have conflicts
# Open those files and look for:
<<<<<<< HEAD
Your changes
=======
Their changes
>>>>>>> main

# 3. Edit the file to keep what you want, remove the markers
# 4. Stage and commit the resolved files
git add conflicted-file.txt
git commit -m "Resolve merge conflict in conflicted-file.txt"
git push origin your-branch
```

---

7. Best Practices

Before You Start Working

- ✅ Always pull the latest changes from `main`
- ✅ Create a new branch for your task
- ✅ Make sure you're on the right branch

While Working

- ✅ Commit often with clear messages
- ✅ Test your code before pushing
- ✅ Keep commits focused on one thing
- ✅ Don't commit sensitive data (passwords, API keys)

Before Submitting

- ✅ Pull latest changes and resolve conflicts
- ✅ Test your code one more time
- ✅ Write a clear PR description
- ✅ Request reviews from teammates

Communication

- 💬 Discuss who's working on what to avoid conflicts
- 💬 Use PR comments to ask questions
- 💬 Be respectful and constructive in code reviews
- 💬 Update your team when you merge changes

---

8. Common Commands Reference

Getting Information

```bash
git status              # See what's changed
git log                 # See commit history
git log --oneline       # Compact commit history
git diff                # See unstaged changes
```

Working with Changes

```bash
git pull origin main        # Get latest changes
git add filename            # Stage a file
git commit -m "message"     # Commit staged changes
git push origin branch-name # Push to GitHub
```

Branch Management

```bash
git branch                  # List branches
git checkout branch-name    # Switch branches
git checkout -b new-branch  # Create and switch to new branch
git branch -d branch-name   # Delete local branch
git push origin --delete branch-name # Delete remote branch
```

Undoing Mistakes

```bash
git restore filename        # Discard changes in working directory
git reset HEAD~1            # Undo last commit (keep changes)
git reset --hard HEAD~1     # Undo last commit (discard changes)
```

---

9. Common Scenarios

"I committed to the wrong branch!"

```bash
git log                     # Copy the commit hash
git checkout correct-branch
git cherry-pick commit-hash
git checkout wrong-branch
git reset --hard HEAD~1
```

"I need to update my branch with main"

```bash
```

```
git checkout main
git pull origin main
git checkout your-branch
git merge main          # Or: git rebase main
```

"Someone pushed changes and now I can't push"

```bash
git pull origin your-branch  # Pull and merge their changes
# Resolve any conflicts
git push origin your-branch
```

---

10. GitHub Desktop Alternative

If command line feels overwhelming, try **GitHub Desktop** (desktop.github.com):

- Visual interface for all git operations
- Easy conflict resolution
- Great for beginners
- Still teaches you git concepts

Quick Start Checklist for New Projects

- [ ] Clone the repository
- [ ] Create a branch for your task
- [ ] Make your changes
- [ ] Commit with clear messages
- [ ] Push your branch
- [ ] Create a pull request
- [ ] Get it reviewed
- [ ] Merge to main
- [ ] Delete your branch
- [ ] Pull the updated main

Getting Help

- 🔗 GitHub's own guides: https://docs.github.com
- 🔗 Git documentation: https://git-scm.com/doc
- 💡 When stuck, ask your teammates!
- 💡 Google the error message
- 💡 Use `git status` when confused

Remember: Everyone makes mistakes with Git. It's a learning process, and there's almost always a way to fix things!